

UNIVERSIDAD NACIONAL DE LA MATANZA

Departamento de Ingeniería e Investigaciones Tecnológicas
Ingeniería Informática
Gestión Aplicada al Desarrollo de Software II (3665)

TRABAJO PRÁCTICO N.º 6

Arquitectura del Sistema

Equipo: **Summate**

Integrantes

Apellido	Nombre	DNI	Correo electrónico
Agasi	Alejo	*43034043*	*aagasi@alumno.unlam.edu.ar*
Huergo	Estefanía	*44787976*	*ehuergo@alumno.unlam.edu.ar*
Naspleda	Julián	*44391303*	*jnaspleda@alumno.unlam.edu.ar*
Panigazzi	Agustín	*43744593*	*apanigazzi@alumno.unlam.edu.ar*
Rios	Cristian	*40015557*	*crrios@alumno.unlam.edu.ar*

Repositorio del equipo

https://github.com/gadsii-unlam/gadsii-summate

MVP funcionando

https://donde-queda.donde-queda.workers.dev

Fecha de entrega: 10/11/2026

Índice

1. Arquitectura real del MVP 3

2. Las decisiones heredadas 4

3. La arquitectura de la gran visión 5

4. Las tecnologías que agregaría la visión 5

5. El componente más riesgoso 6

1. Arquitectura real del MVP

El MVP es una **SPA** (Single Page Application: una sola página que se carga una vez y cambia de pantalla sin recargar) hecha con React 19, TypeScript y Vite, **sin backend**. Todo el cálculo ocurre en el teléfono: la ruta se calcula con **A*** (un algoritmo que busca el camino más corto estimando cuánto falta para llegar) sobre un grafo de 660 nodos y 690 aristas que se descarga como JSON. Se publica como archivos estáticos en Cloudflare Workers y se instala como **PWA** (Progressive Web App: una web que se instala en el celular como una app y puede funcionar sin conexión).

Diagrama 1 — Arquitectura real del MVP «¿Dónde queda?» (tal como está hoy)

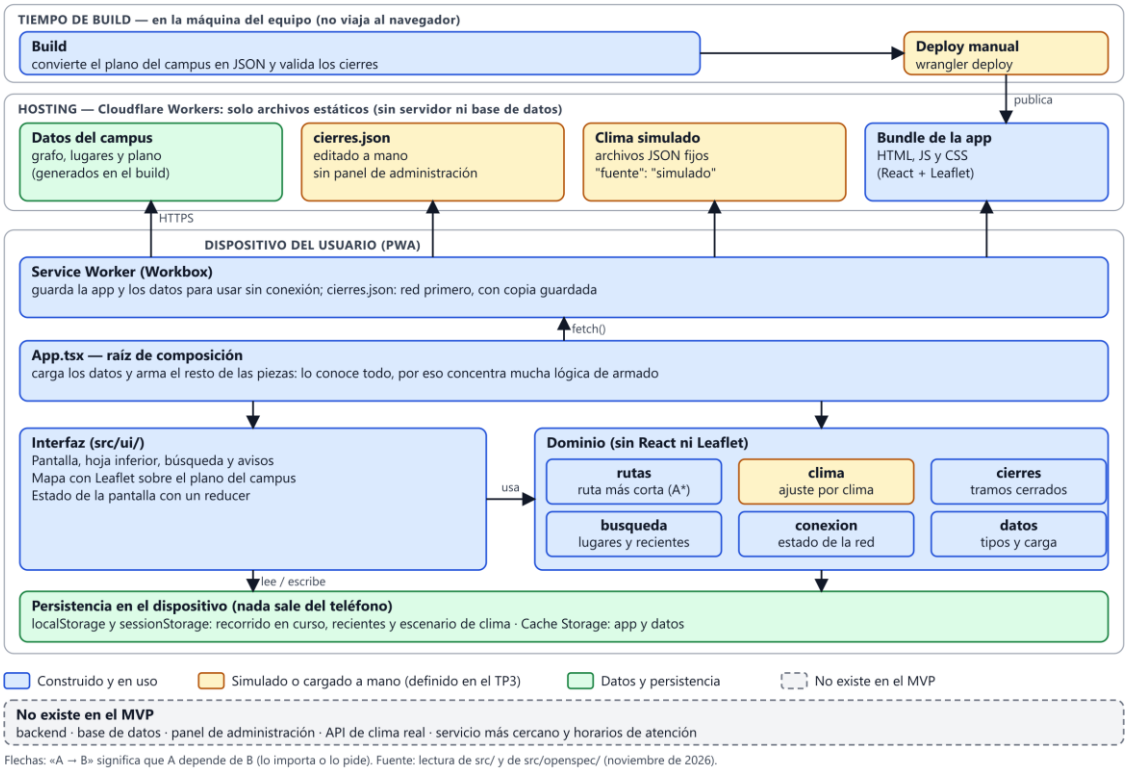


Diagrama 1 — Arquitectura real del MVP (azul: construido; ámbar: simulado o manual; verde: datos).

- **Componentes.** App.tsx carga los datos y arma las piezas. La interfaz (ui/) dibuja la pantalla, el mapa y la búsqueda. El dominio (rutas, clima, cierres, búsqueda, conexión, datos) contiene la lógica y no usa React ni Leaflet: lo impone una regla de ESLint. El service worker (el componente de la PWA que guarda archivos para usar sin conexión) decide qué se conserva en el teléfono.
- **Dependencias.** Las flechas van de arriba hacia abajo: App.tsx depende de la interfaz y del dominio, y la interfaz depende del dominio, nunca al revés.
- **Capas.** Build (en la máquina del equipo, no viaja al navegador), hosting estático (Cloudflare Workers) y dispositivo del usuario.
- **Datos.** El grafo, los lugares y el plano son archivos estáticos generados en el build a partir de mapa-unlam.svg. No hay base de datos: lo único que se guarda es en el teléfono (localStorage, sessionStorage y la caché del service worker).
- **Construido vs. simulado (TP3).** Está construido el motor de rutas, la interfaz y la exclusión de tramos cerrados. Está simulado el clima (archivos JSON fijos) y la carga de cierres, que se edita a mano en cierres.json y se publica con un deploy, sin panel de administración.

2. Las decisiones heredadas

Decisión	¿Quién la tomó?	¿La sostendrían hoy?	Dato del TP2 o del TP5
Sin backend: los datos del campus y los cierres son archivos estáticos publicados con cada deploy.	**La herramienta.** Claude Code (Opus 5.5) con OpenSpec la propuso junto con las demás decisiones estructurales (docs/prompts.md, línea 20; commit 113f1c1, cuyo mensaje dice «SPA sin backend»). El equipo había decidido en el TP3 no construir panel de administración y precargar los cierres; la herramienta eligió el mecanismo (JSON en el repositorio + deploy).	**Para el MVP, sí. Para la visión, no.** Es lo más simple que cumple lo que había que probar, pero no alcanza para que alguien que no toca el código cargue un cierre.	**Sostiene:** en el TP2, 5 de 5 encuestados tienen «datos móviles con señal irregular»; con todo estático y cacheable, el MVP no depende de un servidor para mostrar la ruta. **Contradice parcialmente:** en el TP5 el cierre de T2 hubo que cargarlo en cierres.json el día anterior, validarlo, publicar y, al terminar, retirarlo y volver a publicar. Un cierre real no puede esperar a un deploy.
La composición está concentrada en App.tsx (512 líneas): hace los fetch, arma el orquestador, el monitor de conexión y los cierres, restaura el recorrido y provee 5 contextos. Algunos componentes además llaman a funciones de dominio.	**Nadie, salió así.** No hay una decisión de diseño escrita: la herramienta propuso la separación en carpetas de dominio, pero la concentración en un solo archivo es el resultado de ir sumando cambios de OpenSpec uno tras otro.	**Parcialmente.** El dominio está bien aislado y es testeable, pero App.tsx es el único lugar que lo conoce todo y cada funcionalidad nueva (usuarios, panel, sincronización) lo va a tocar.	**Ningún dato del TP2 ni del TP5 la sostiene ni la contradice.** Nadie en la prueba pudo notarla: es una decisión de comodidad de la construcción, no de los usuarios.

3. La arquitectura de la gran visión

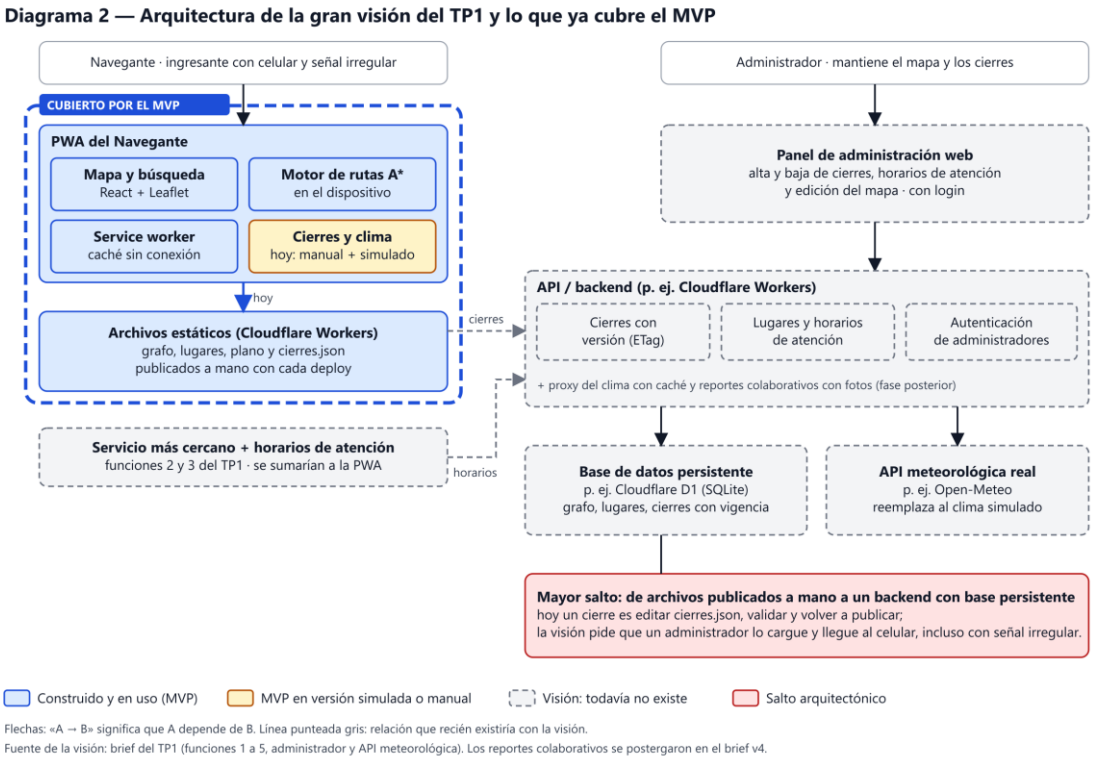


Diagrama 2 — Gran visión del TP1; el recuadro azul marca lo que cubre el MVP.

El producto completo del TP1 tiene cinco funciones: (1) mapa y ruta más corta, (2) estado de atención de los puntos de interés según día y hora, (3) búsqueda del servicio más cercano, (4) administración de caminos cerrados por un administrador y (5) rutas adaptadas al clima con una API meteorológica. Arquitectónicamente pide dos clientes (el Navegante, que ya existe, y un panel de administración), un backend con autenticación y una base de datos. El recuadro azul marca lo que cubre el MVP: la función 1 completa y las funciones 4 y 5 en versión manual o simulada. Las funciones 2 y 3 quedan fuera porque no existen todavía.

El salto arquitectónico más grande es pasar de archivos estáticos publicados a mano a un **backend con base persistente y un administrador que cargue los datos**, sin perder lo que hoy funciona: que la ruta se calcule en el dispositivo y siga a la vista con señal irregular. Todo el MVP asume que los datos cambian solo cuando el equipo publica una versión nueva; la visión asume que cambian durante el día (un cierre, el horario de un servicio) y que ese cambio llega a un celular con señal intermitente.

4. Las tecnologías que agregaría la visión

- **Cloudflare Workers + D1 (SQLite)** como backend y base de datos, para guardar cierres con vigencia, horarios y administradores. La elegimos por continuidad: el MVP ya está desplegado en Workers, así que no hay que sumar un servidor ni otra infraestructura para mantener. Ningún TP anterior relevó restricciones de infraestructura de la UNLaM, así que no la justificamos con el contexto de la universidad sino con la continuidad.
- **API meteorológica real** (por ejemplo Open-Meteo). Reemplaza al servicio simulado; el código ya tiene la costura (src/clima/servicio-simulado.ts es lo único que habría que cambiar). Hoy la promesa del TP1 de adaptar la ruta al clima real se prueba solo con archivos fijos.
- **Sincronización de cierres entre el servidor y la PWA.** Cada versión de los cierres lleva una etiqueta (el ETag de HTTP); la app la compara al abrirse y descarga solo si cambió, apoyándose en el service worker que ya existe. **Se justifica contra el contexto de uso del TP2:** 5 de 5 encuestados usan datos móviles con señal irregular, así que un cierre cargado por el

administrador tiene que llegar aunque la conexión sea mala, y la app tiene que seguir funcionando con los últimos datos si no llega. Por eso no depende de la sincronización en segundo plano, que no está disponible en todos los navegadores (en la prueba del TP5 hubo un iPhone y dos Android).

5. El componente más riesgoso

El subsistema de cierres (src/cierres/, cierres.json y la estrategia NetworkFirst del service worker, que intenta primero la red y recién después usa la copia guardada).

- **Ya mostró fricción en la prueba del TP5.** En T2 el cierre estaba cargado y el desvío se calculaba bien, pero dos de tres usuarios no lo percibieron hasta llegar a la cinta: U2 se detuvo a unos 3 m («Uy, esto está cerrado») y U3 llegó hasta la cinta («No me fijé en lo rojo hasta que ya estaba ahí»). U2 pidió «que me avisara más fuerte lo del cierre». Solo U1 lo vio en el mapa antes de caminar.
- **Es el dato que más va a cambiar y el que más daña si está viejo.** Hoy un cierre es un archivo editado a mano y un deploy. Con señal irregular (el caso normal según el TP2), si la red tarda más de 4 s la app usa la última copia guardada o, si nunca se guardó, la que vino con el build. El usuario puede ver cierres desactualizados sin enterarse.
- **No escala sin rediseño.** Hay un solo archivo, un solo editor y ningún historial de quién cerró qué. Con varios administradores y cientos de usuarios, el cuello de botella no es el cómputo (la ruta se calcula en cada teléfono) sino que el dato llegue fresco y que el usuario lo vea.